

Product Intelligence for Industrial Commerce

An enrichment engine that refuses to guess.

A wrong specification ships a broken machine. So every value carries its evidence, and the engine leaves a field blank, flags the contradiction, or blocks publication rather than emit a number it cannot defend.

2.75×

attribute coverage lift

0.0%

contradictions on
evidence-backed values

100%

seeded defects caught
(51 / 51)

\$0.0084

per SKU, batched

Live prototype industrial-product-intelligence.onrender.com · 102-case benchmark reproduces from a clean clone



unilog | IIS

UniHack

AI-Powered Product Intelligence for
Industrial Commerce

Team Details

Team name: Team Codes

Team leader: Ankur Goswami

Members: Ashwin S · Argho Kumar Halder · SV Chiranjeevi

Brief about your solution

Given a messy catalogue row, produce a complete, standardised, search-ready product record — with every value traceable.

The engine is deterministic first. AI is a bounded contributor, never the source of record.



One pipeline. A value read off page 2 of a datasheet earns the same provenance, validation and score as a hand-typed one.

Every value is attributable

Six provenance classes, ranked. When two stages disagree the stronger wins, the conflict is recorded, and the winner's confidence is reduced.

Refusal is a feature, not a failure

Outside the approved vocabulary, not an exact 64th, no manufacturer source — the engine says so and blocks, instead of guessing plausibly.

Written the customer's way

The same product rendered five times to five character limits, reproduced exactly against Unilog's own labelled rows.

How the solution enriches, proves and scales

1

Enriching minimal input

Part number and brand decode against ISO 15 / ISO 898-1 and a curated knowledge base; datasheets, product pages and catalogue rows all feed the same pipeline.

2.75×
coverage lift

61.1%
withheld values recovered

2

Accuracy and trust

16 cross-field engineering rules catch contradictions that are individually plausible. Readiness scores on three axes and returns publish / review / blocked.

100%
defects caught (51/51)

0.0%
false alarms on clean records

3

Scale

611 rows/s ingested and 305 products/s enriched on one core. Unseen categories are learned from the data, not hand-curated.

8.9% → 81.7%
their 1,000 rows — 38 of
83 clusters approved

\$0.0084
per SKU, batched

Opportunities — and what makes this different

Coverage is cheap. Any system reaches 100% by inventing everything. The hard part — and the only part a distributor can actually publish — is precision you can defend.

The USP

It is the only enrichment engine we know of that reports what it refused to do. Every AI proposal, vocabulary mismatch and rejected source is listed with its reason — on screen and in the export.

Proven, not asserted

A live ablation measured the AI making accuracy worse; bounding it to gap-filling beat both the raw model and the engine alone. An independent A–Z pass then found five defects — all fixed, each with a regression, every benchmark figure unchanged.

Measured against Unilog's own labelled delivery rows

14 / 14

prose fields exact — given the attribute values

252

delivery columns emitted in their exact order

61.5%

precision on the fields we do answer

88 of 114

left blank: the data is not in the input row

List of features offered by the solution

1

Provenance on every value

Six ranked classes from supplied to defaulted, with evidence text and a source URL.

2

Cross-field validation

16 rules, plus a part number's own standard challenging a supplied value it contradicts.

3

Bounded AI gate

The model may fill a gap or displace a default. It may never overrule evidence.

4

Unilog compliance layer

Approved UOM, exact-64th fractions, list-of-values refusals, character limits — on screen.

5

Auto-taxonomy learning

Clusters unclassified rows and proposes a schema; a human approves it.

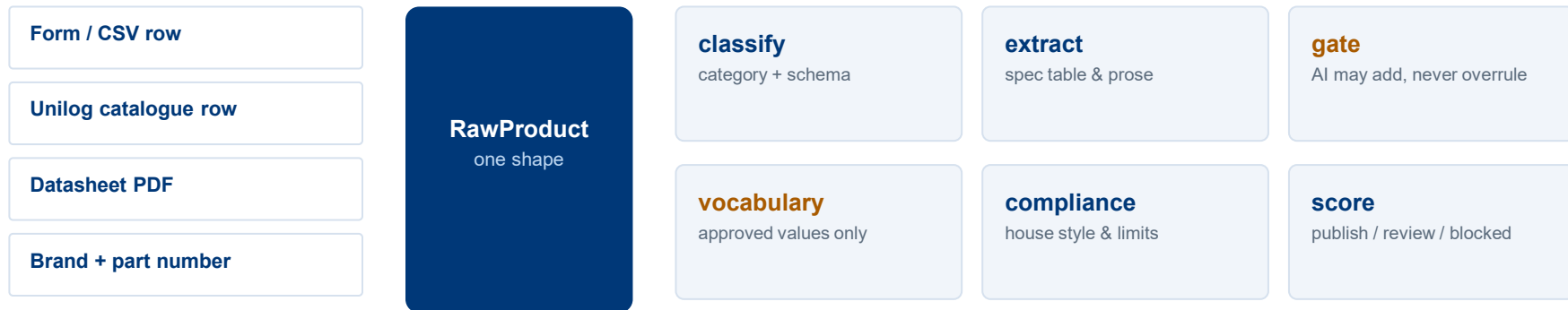
6

Manufacturer-only sourcing

Marketplaces, retailers and distributors refused — each with a stated reason.

Process flow diagram or Use-case diagram

Four input paths converge on one RawProduct, then one pipeline. Refusal points are marked in amber.



Output EnrichedProduct → 252-column Unilog delivery sheet · schema.org JSON-LD · catalogue CSV. Every attribute carries provenance, confidence, evidence and the URL it was read from.

At each amber point the engine can stop. Across the 102-case benchmark the gate refused 28 of the model's 95 proposals; a value outside the approved list keeps its original and raises a violation rather than being rewritten to the nearest match.

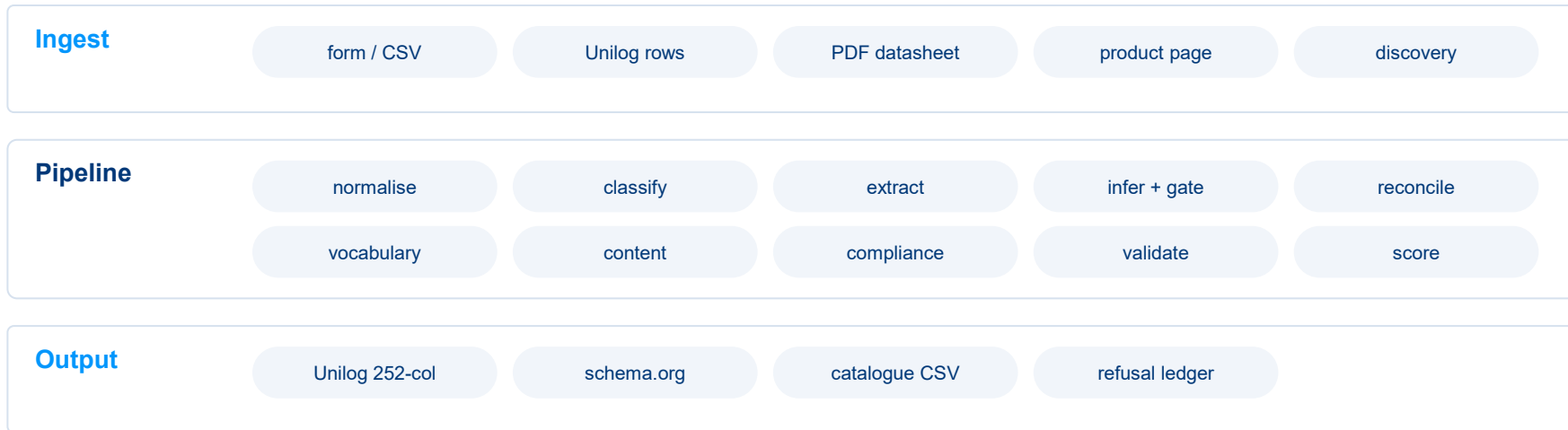
Their expected output, next to ours

Two fully worked delivery rows — the only labelled ground truth in their pack. Row 1 below, their value then ours. Every prose field is built by formula against their character limits, never free-hand.

FIELD	UNILog EXPECTED	OURS
INVOICE	DISHWASHER LEG 5 SST 120V 15A 50-1/4IN	identical — character for character
MOBILE	Rheem Manufacturing FRIGIDAIRE, Dishwasher, Professional Series, PDSH4816AF	identical — character for character
SHORT	FRIGIDAIRE® Professional Series PDSH4816AF Dishwasher With CleanBoost™, Leg Mounting, 5-Wash Cycle, Stainless Steel	identical — character for character
RETAIL	Professional Series Dishwasher, Leg Mounting, 5-Wash Cycle, Stainless Steel	identical — character for character

14/14 exact when the attribute values are supplied. 2/14 from the six-column input row alone — one engine, two different inputs. Series, Mounting, Wash Cycles and Voltage are not in a catalogue row; they are on frigidaire.com, which their own delivery row cites as MFR URL. The formatting is solved. The sourcing is not, and 88 of 114 blank fields is the size of it.

Architecture diagram of the proposed solution



Providers are swappable

Deterministic engine (free, default) · Claude tool-use (live) · Hybrid, where the model may only fill a gap or displace a default.

The customer's rules are data, not code

Taxonomy, UOM table, list of values, export schema and source policy are JSON. Their spreadsheets land as a data drop, not a rewrite.

No product database by design — their PIM stays the system of record. The engine takes a row and returns an enriched row.

Technologies used in the solution

Engine

Python 3.13	FastAPI
Pydantic v2	pdfplumber
BeautifulSoup + lxml	httpx

Interface

React 19	Vite
zero runtime UI dependencies	design tokens, dark mode

AI

Claude Sonnet	tool-schema structured output
server-side web search	content-addressed cache

Delivery

Docker (single service)	Render
UptimeRobot	GitHub

Deliberate constraint the deployed instance holds no API key and cannot spend. Reviewers see genuine model output from 20 pre-computed records at zero cost.

Cost per SKU — measured, then projected

Their baseline: 10 minutes of analyst time per SKU at \$35/hr.

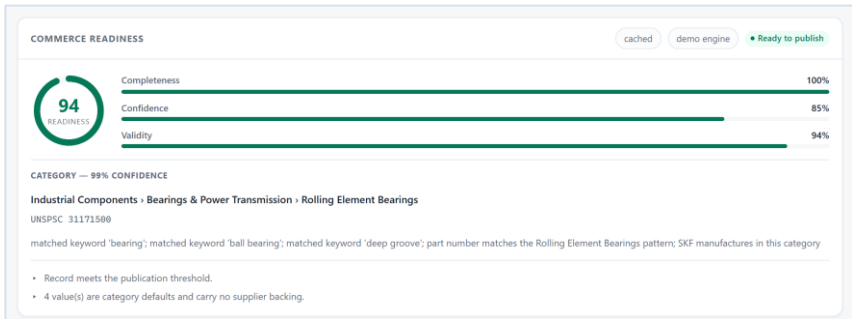
Manual, today	\$5.83	per SKU	\$4,372,500
Every SKU to the model	\$0.02381	standard rates	\$17,857
+ deterministic-first triage	\$0.01681	70.6% call rate	\$12,605
+ Batch API	\$0.00840	projected	\$6,302

0.14% of manual cost at 750,000 SKUs/month — the organizers' own scaling target.

Binding constraint is the API rate limit, not compute: 0.7 compute hours/month, 0.20 calls/second sustained.

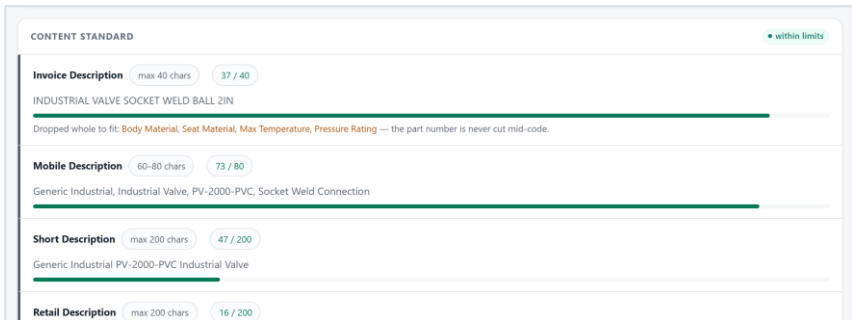
Snapshots of the MVP

Captured from the live prototype — the same link a judge opens.



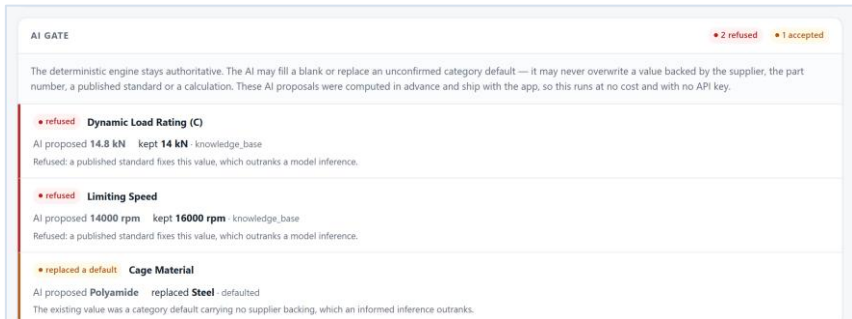
Single product enrichment

Readiness on three axes, category with its reasoning, provenance per value



The customer's content standard

Five formats to five limits; the 40-character line names the facts it dropped



The AI gate refusing

Two proposals refused against ISO 15; one unbacked default replaced



Catalog at volume

10 products in under a second — 7 publishable, 1 review, 2 blocked

What we measured but did not solve

Each item below is a known gap with a number attached, not a wish list.

Finding the right manufacturer URL

We built headless rendering and measured it: still zero of four. Two URLs are simply wrong, and Milwaukee's page renders fine with no spec table on it. The constraint is search, not rendering — and search costs money per SKU.

The seven reference files still to land

UOM standards, content guidelines, the 161k-row list of values and the 27k-row brand list. Every stub reports source: provisional, and swapping them in is a data drop — the seams are already built.

Attribute recall on categories without a standard

41.7% against 99.5% where ISO fixes the answer. Honest ceiling of a knowledge base; closing it needs manufacturer documents, not a better prompt.

Durable storage for runtime state

Results are content-addressed and reproduce exactly, but approved categories sit on a container disk the free tier does not persist. A mounted volume or Postgres.

Batch API and video extraction

Costed at \$0.0084/SKU but not implemented, and flagged by the organizers as innovative but slow. Both are catalogue-path work, not interactive.

Links

Working prototype

<https://industrial-product-intelligence.onrender.com>

Live, monitored, no API key needed — every tab is explorable at zero cost.

GitHub repository

<https://github.com/ashwinsureshh/industrial-product-intelligence>

Public. 10 test suites, all free to run. Benchmarks reproduce from a clean clone.

Demo video (3 min)

<https://youtu.be/AUagw9KwWfE>

Walkthrough of enrichment, the AI gate refusing two values, and the delivery export.

Reproduce the headline: `cd backend && python run_benchmark.py` (102 cases, no API key, \$0)

unilog | **H2S**
HACK2SKILL

UniHack

AI-Powered Product Intelligence for
Industrial Commerce

Thank You

